

目录

1. 芯片上电之后发生了什么？一文读懂 Boot Simulation

1.1 Boot Simulation 到底是什么

1.1.1 真实固件

1.1.2 真实 RTL

1.1.3 从真实 Reset Vector 开始

1.1.4 固件驱动，testbench 观测

1.2 Boot Simulation 填补了哪一块验证空白

1.3 为什么模块都通过，芯片仍可能启动失败

1.3.1 第一层：验证启动正确性

1.3.2 第二层：验证系统集成正确性

1.3.3 第三层：风险前移与签核

1.4 一套 Boot Simulation 怎么跑起来

1.4.1 第一步：构建可信输入

1.4.2 第二步：建立真实启动环境

1.4.3 第三步：让启动链路真实执行

1.4.4 第四步：形成证据链

1.5 怎么判断 Boot Simulation 真的通过了

1.5.1 核心启动证据矩阵

1.5.2 扩展检查清单

1.6 结语

1. 芯片上电之后发生了什么？一文读懂 Boot Simulation

来源：<https://mp.weixin.qq.com/s/aBrgD1Q6nQDnlgz41jZ5WA>

作者：SJ66

update 2026/08/17 23 : 17

技术博客

如果 BootROM、Clock、Reset、NoC 和存储控制器分别都通过了自己的模块级验证，是否就能证明整颗芯片一定可以启动？

答案是否定的。

一颗芯片从上电到真正“活起来”，需要走过一条很长的启动链路：

1. Power / Clock

电源与参考时钟稳定

2. Reset Release

复位按顺序释放

3. Strap / Fuse Latch

启动配置锁存

4. CPU Reset Vector

CPU 从固定地址取指

5. BootROM Execute

BootROM 开始执行

6. Boot Media Init

初始化启动介质

7. Image Read

读取下一阶段镜像

8. Authentication

镜像认证与完整性检查

9. Load to Memory

镜像加载到目标内存

10. Jump to Next Stage

控制权交给下一阶段软件

这条链路中，任何一个环节出现问题，都可能导致芯片无法启动。

CPU 可能根本取不到第一条指令，也可能停在 **BootROM** 的某个轮询循环中，还可能在访问启动介质、认证镜像或跳转到下一阶段软件时突然卡死。

更棘手的是，**BootROM** 属于芯片最早执行的软件之一。一旦相关逻辑在流片后固化，修改代价通常非常高。

于是，一个关键问题摆在验证团队面前：

在芯片还没有流片之前，如何证明真实启动固件能够在真实的 SoC 硬件路径上，把芯片从复位状态带到下一阶段软件？

这正是 Boot Simulation 要回答的问题。

1.1 Boot Simulation 到底是什么

严格来说，Boot Simulation 是指：

让一份真实或具有代表性的候选启动固件，从芯片真实的复位入口开始，在 SoC RTL 上由 CPU 完成真实的取指、译码和执行，驱动相关硬件完成从复位到约定启动终点的完整过程。

这个定义中有四个关键词。

1.1.1 真实固件

Boot Simulation 使用的应尽量是发布候选版本或具有代表性的启动固件，而不是专门为了让仿真快速通过而大幅简化的替代代码。

如果固件为了仿真而绕过了某些硬件初始化、缩短了关键等待流程，甚至由 testbench 直接代替它配置寄存器，那么最终“跑通”的链路就不再等同于真实启动链路。

1.1.2 真实 RTL

固件不是运行在抽象的软件平台模型上，而是在当前版本的 SoC RTL 上运行。

CPU 的取指、NoC 事务、寄存器访问、存储控制器响应和安全模块行为，都来自实际的 RTL 实现。

1.1.3 从真实 Reset Vector 开始

CPU 应当从架构规定的 Reset Vector 获取第一条指令，而不是由测试平台直接将 CPU 放到某个已经完成初始化的中间状态。

只有从真实复位入口开始，才能覆盖：

- Clock 和 Reset 的释放顺序
- Strap 和 Fuse 的锁存
- BootROM 地址映射
- CPU 首次取指
- 启动早期的 NoC 和外设访问

1.1.4 固件驱动，testbench 观测

Boot Simulation 与许多模块级 UVM 测试最本质的区别，在于激励的主要来源发生了变化。

在模块级验证中，sequence 通常直接配置寄存器、发起总线事务并检查 DUT 响应。

而在 Boot Simulation 中：

- 固件负责配置寄存器
- 固件负责初始化外设
- 固件负责读取启动介质

- 固件负责解析、认证和加载镜像
- 固件负责执行最终跳转

测试平台负责提供时钟、复位、启动介质模型和外围环境，并根据需要注入异常、采集日志、记录波形和判断结果。

可以用一句话概括：

在 Boot Simulation 中，固件是主角，测试平台是配角。

1.2 Boot Simulation 填补了哪一块验证空白

Boot Simulation 不是唯一的启动验证方法。

UVM 仿真、ISS/QEMU、RTL 仿真、Emulation 和 FPGA 原型验证，都能从不同角度发现问题。

它们之间的主要区别如下。

UVM/IP 仿真

单个模块或子系统 RTL

速度 · 较慢可观测性 · 高

主要回答：模块功能和接口协议是否正确

ISS/QEMU

指令集或平台行为模型

速度 · 快可观测性 · 中

主要回答：软件栈能否在抽象平台上运行

RTL Boot Simulation

SoC RTL 与真实固件

速度 · 慢可观测性 · 高

主要回答：固件能否驱动真实 RTL 完成启动

Emulation

映射到硬件加速平台的设计

速度 · 较快可观测性 · 中

主要回答：更长的启动流程和软件负载能否运行

FPGA Prototype

映射到 FPGA 的设计

速度 · 快可观测性 · 较低

主要回答：系统能否以接近真实速度运行较长软件

Boot Simulation 的独特价值，并不是速度最快，也不是能够运行最长的软件负载，而是同时具备两个特点：

1. 1.

固件看到的是当前版本的真实 RTL 行为。

2. 2.

验证工程师仍然能够观察 PC、寄存器、总线事务、状态机和波形。

因此，它特别适合发现由以下因素引起的启动问题：

- 固件假设与 RTL 实现不一致
- 地址映射配置错误
- Clock 与 Reset 顺序错误
- 启动早期 NoC 访问异常
- 外设控制器就绪时序不匹配
- Fuse 或 strap 配置组合错误
- 镜像认证与加载路径异常

但 Boot Simulation 也有明确边界。

它能够证明的是：在当前 RTL、当前固件和给定配置组合下，存在一条具有可验证证据的启动路径。

它不能替代模块级验证去覆盖每一个接口协议细节，不能替代形式化验证穷尽所有状态空间，也不能替代硅后验证发现真实工艺和电气环境中的问题。

1.3 为什么模块都通过，芯片仍可能启动失败

Boot Simulation 的价值可以归纳为三个层次。

1. 启动正确性

芯片是否能够完成基本启动

2. 系统集成正确性

软硬件与跨模块契约是否成立

3. 风险前移与签核

在流片前发现问题并形成证据

1.3.1 第一层：验证启动正确性

启动是一切后续功能的基础。

Boot Simulation 首先需要证明：

- CPU 能够从正确的 Reset Vector 取指
- BootROM 能够正常执行
- 启动介质能够完成初始化和读取
- 镜像能够被定位、认证和加载
- CPU 能够跳转到下一阶段软件

如果最基础的启动链路不可信，后续所有性能、功耗和系统功能验证都建立在不稳定的基础上。

1.3.2 第二层：验证系统集成正确性

模块级验证通过，并不代表系统集成后一定能够正常启动。

因为模块级环境通常独立验证单个模块的功能，而真实 Boot 流程会在特定时序下，把多个模块串成一条因果链：

1. CPU 执行 BootROM
2. BootROM 配置 Clock 和外设
3. CPU 通过 NoC 访问存储控制器
4. 存储控制器访问启动介质
5. 数据经过互连写入目标内存
6. CPU 跳转到加载后的镜像

这条链路中的问题，往往不属于某一个单独模块，而是来自模块之间的隐含假设。

1.3.2.1 一个典型问题

假设 NoC 和存储控制器分别通过了模块级验证。

在 SoC 集成后，BootROM 检测到存储控制器的初始化完成标志，于是立即发起第一次介质读取。

但此时控制器虽然已经报告“初始化完成”，内部数据通路或介质状态机却尚未真正进入可服务状态。

现象

- CPU 长时间停留在某段轮询代码中
- 启动日志停止在介质初始化之后
- 仿真无法进入镜像加载阶段

根因

BootROM 将“初始化完成标志”理解为“控制器已经可以响应读取”，但 RTL 对该标志的定义可能只是“初始化流程已经退出”。

软件和硬件对同一个状态的理解不一致。

可观察证据

- PC 在一个固定地址区间反复执行
- CPU 已经发出读取请求
- NoC 请求已经到达控制器
- 控制器没有在预期窗口内返回完成响应
- 可见寄存器状态与内部状态机不同步

这个问题可能无法在单独的 NoC 测试或存储控制器测试中暴露，却会在真实固件形成完整访问序列时出现。

1.3.3 第三层：风险前移与签核

Boot Simulation 还能将原本可能在硅后 Bring-up 阶段才暴露的问题，提前到流片之前发现。

除了正常启动路径，还应覆盖：

- 镜像认证失败
- 启动介质读取超时
- 非法镜像或镜像损坏
- 异常向量触发
- 恢复模式或下载模式
- Warm Reset 后的状态清理

通过这些场景，可以为 BootROM 固化和流片签核提供可追溯的验证证据。

1.4 一套 Boot Simulation 怎么跑起来

一套完整的 Boot Simulation，可以抽象为四个步骤。

1. 构建可信输入
2. 建立真实启动环境
3. 从 Reset Vector 执行
4. 收集并判断启动证据

1.4.1 第一步：构建可信输入

仿真结论是否可信，首先取决于输入是否可信。

通常需要准备：

- 当前待验证的 SoC RTL
- 真实或发布候选级 BootROM
- Fuse 和 strap 配置
- 启动介质内容
- 下一阶段启动镜像

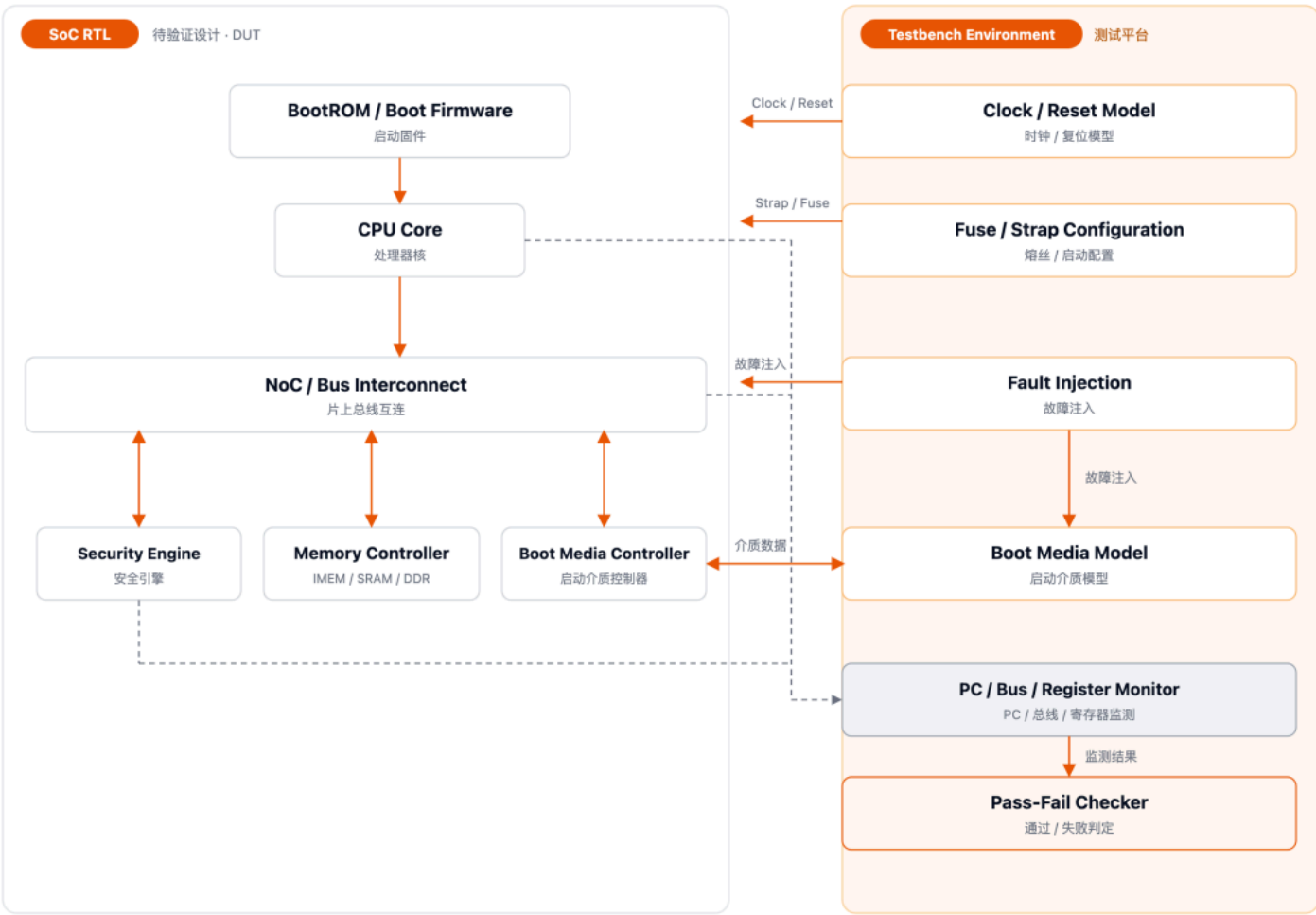
- Clock 和 Reset 配置
- 必要的内存与外围设备模型

关键原则是尽量减少仅为仿真而进行的特殊修改。

固件越接近最终发布版本，配置越接近真实芯片场景，仿真结论对流片决策的参考价值就越高。

1.4.2 第二步：建立真实启动环境

一个典型的 Boot Simulation 环境可以抽象为：



Boot Simulation 环境框图：SoC RTL 内部模块与 Testbench Environment 组件的连接关系

在这个环境中，testbench 可以提供启动介质模型，但介质初始化、读取命令、镜像解析、认证和加载动作，应尽量由固件真实触发。

1.4.3 第三步：让启动链路真实执行

仿真启动后，应让 CPU 从真实 Reset Vector 开始执行。

不要由 testbench 直接完成以下动作：

- 跳过 BootROM 初始化
- 直接写入目标寄存器状态
- 直接将镜像放入最终执行地址
- 直接修改 PC 到下一阶段入口
- 绕过真实的认证或加载过程

测试平台可以做的是：

- 提供外部环境
- 设置 Fuse 与 strap
- 模拟启动介质
- 注入认证失败或读取超时
- 采集日志、波形和总线事务
- 根据证据判断测试结果

1.4.4 第四步：形成证据链

一个可信的 Boot Simulation 结果，应该能够回答：

- CPU 是否从正确地址取到了第一条指令？
- 固件是否读取到了正确的启动配置？
- NoC 事务是否到达了预期目标？
- 启动介质是否完成了初始化？
- 镜像数据是否被正确读取和写入内存？
- 认证成功和失败路径是否符合预期？
- 最终跳转地址是否正确？
- 跳转后是否取到了下一阶段的第一条指令？

只有这些问题都有明确证据，才能说明启动链路确实完成，而不是仿真仅仅“没有报错”。

1.5 怎么判断 Boot Simulation 真的通过了

Boot Simulation 最容易出现的误区，是把“仿真跑完”当成“验证通过”。

事实上，以下现象都不能单独构成充分的通过条件：

- 仿真器没有报 fatal
- CPU 仍然在执行指令
- 日志中出现了某个成功字符串
- 波形运行到了预定时间
- testbench 没有触发 timeout

真正有说服力的通过标准，是关键启动节点都有可追溯、可复现的证据。

1.5.1 核心启动证据矩阵

Clock / Reset— 时钟不稳定或复位释放顺序错误

通过证据：时钟稳定标志、复位释放时序波形

Strap / Fuse— 启动配置错误，固件进入错误分支

通过证据：锁存后的配置值、固件读取日志

Reset Vector— CPU 从错误地址取指，或无法取得有效指令

通过证据：首次 PC 轨迹、首次取指总线事务

NoC / Boot Media— 总线访问超时、控制器未就绪、读取内容错误

通过证据：总线事务日志、控制器状态、数据比对

Secure Boot— 合法镜像被拒绝，或非法镜像被接受

通过证据：认证结果、校验日志、失败后的安全状态

Image Load / Jump— 镜像不完整或跳转地址错误

通过证据：内存数据比对、跳转 PC、下一阶段首次取指

1.5.2 扩展检查清单

根据具体 SoC 的启动方案，还可以继续检查：

- BootROM 地址映射与内容完整性
- GPT、镜像头或其他标准格式解析
- 多种启动介质选择
- 启动介质读取超时
- 异常向量处理
- 恢复模式或下载模式
- Watchdog 触发
- Warm Reset
- 复位前后的状态清理
- 多组 Fuse 与 strap 配置组合

并不是每一颗 SoC 都需要覆盖完全相同的对象。

真正重要的是，每个测试场景都应该明确三个问题：

1

要验证什么？

Verification Objective

2

可能怎样失败？

Failure Mode

3

什么证据证明正确？

Observable Evidence

这三个问题共同构成 Boot Simulation 的最小验证闭环。

1.6 结语

Boot Simulation 验证的，不只是某一段 BootROM 代码是否能够执行。

它验证的是从复位释放到控制权交接之间，启动固件与 SoC RTL 的完整契约是否成立：

- CPU 是否从正确入口开始执行
- Clock、Reset 和配置是否符合固件假设
- NoC、存储控制器与启动介质是否能够协同工作
- 镜像是否被正确读取、认证和加载
- 正常与异常路径是否进入预期状态
- 控制权是否被正确交给下一阶段软件

因此，真正可信的启动签核标准，不应只是：

仿真跑完了，并且没有报错。

而应该是：

从 Reset Vector 到下一阶段镜像的每一个关键节点，都留下了可复现、可追溯并能够支撑判断的工程证据。